Four rules for intuitionistic primitive recursive arithmetic

Substitution at witness discharge, with one computation schema

Prepared for Vladimir Reshetnikov

November 12, 2026

Abstract

We give a four-family presentation of full first-order intuitionistic primitive recursive arithmetic. Terms use zero and the existing successor-seeded recursor, with its zero-body canonicalization. Formulas are equations and the existing paired binder D. The rules are one successor calculation, clause use, paired introduction, and substitutive witness discharge. There is no fixed reflexivity axiom, separate equality rule, separate existential-elimination rule, or evaluation of iterator bodies by a normalization algorithm.

The new proof is a short, noncircular recovery of equality and ordinary witness elimination. Composed with the pinned repository's canonical recursor and scalar bridges, it gives a whole-domain bi-interpretation with iPRA, actual equality, identity numerical composites, and equiconsistency. A matched five-family checker is reduced from 2318 to 2267 normalized bytes, from 726 to 713 significant tokens, and from 809 to 798 AST nodes. This is a concrete local simplification, not an absolute minimum or a new repository-wide size record. The historical 516-byte compact and 788-byte public-goal profiles are not beaten. A separate appendix proves a constancy-only alternative to induction, while charging its additional Boolean-equality axiom explicitly.

Contents

1 Result, baseline, and limits **2**

2 The entire native syntax **2**

3 Exactly four inference families **3**

4 The equality bootstrap is noncircular **4**

5 Exact equivalence with the separate-rule calculus **5**

6 The bi-interpretation with full first-order iPRA **6**

7 The complete checker and the simplicity comparison **8**

8 A boundary result: unary congruence is insufficient **9**

9 Further research questions **10**

A A constant-recurrence rule in place of induction **10**

B Reproduction and artifact map **15**

C Complete selected acceptance module **16**

1 Result, baseline, and limits

The question asks for a presentation simpler than the current `ConciseIPRA` project. We froze the repository at commit `17898ea7e6a1300dfdcad71040d5daeed67916d4`. The inspected state contains 67 filed research packages and six further ZIP archives in `docs/incoming`. In particular, the incoming `witness_discharge` package already has three instruction families and a fixed reflexivity clause, using full computation normalization. That construction is a necessary comparison, not a new result of this report. The existing canonical-recursors package already supplies the successor-seeded term language used here [1, 2, 3].

Our change is to allow a *formula template* in witness discharge, so that the discharged equality may have an arbitrary source. This removes the dependence of the equality bootstrap on normalization of selector packets. Explicit computation is retained, and reflexivity is derived from it. On the canonical-recursors basis, only one explicit computation schema is needed. Thus the complete presentation has four families, no permanent axiom clause, and only a structural canonicalizer.

The comparison with a matched five-family implementation isolates the fusion of equality transport and witness elimination. The stronger claim that this presentation lies below every current complexity frontier is *not established*. It would be misleading to call four families a global minimum when a three-family presentation is already present, or to omit the cost of its canonicalizer. The improvement here is the combined specification and its matched complete checker.

Status of the proofs. The new equivalence has a written proof and executable certificate checks. It has not been newly verified in Lean or Rocq. The repository separately formalizes the initialized-scalar/native-to-iPRA bridge; its successor-seeded canonical presentation is a written result. We use that result with its stated hypotheses and include its complete source for audit. We did not rebuild the upstream Lean project.

2 The entire native syntax

2.1 Terms and the small canonicalization

Start with raw numerical terms

$$t ::= x \mid 0 \mid R(n, a; u.h).$$

Only the body h binds u. Both the counter n and initializer a are outside that binding. Define

$$S(a) := R(0, a; u.0), \qquad 1 := S0.$$

The intended recursor is successor seeded:

$$R(0, a; u.h) = Sa, \qquad R(Sn, a; u.h) = h[R(n, a; u.h)/u].$$

The first equation is made part of the syntax. Recursively normalize all children, and replace the body of any R with literal counter zero by literal zero. Write this structural operation as ν. A term is canonical exactly when every subterm $R(0, a; u.h)$ has $h \equiv 0$.

This operation does not unfold successor iterations, execute a body, evaluate a numerical program, or test whether a general term denotes zero. It preserves a nonzero outer constructor. In particular,

$$\nu(t) \equiv 0 \quad \Longleftrightarrow \quad t \equiv 0.$$

Substitution everywhere in this calculus means capture-avoiding raw substitution followed by ν. We write $F[t/v]$ below with that canonical-substitution convention understood.

Lemma 2.1 (Coherence)**.** *For any finite term or formula T and capture-free substitution σ,*

$$\nu(T[\sigma]) = \nu\big(\nu(T)[\nu \circ \sigma]\big).$$

Canonical substitution is associative, commutes with renaming, and introduces no new free variables beyond those in its arguments.

Proof. Use structural induction after freshening bound variables. At an R-node, if the counter was zero, both sides erase the body. If a nonzero counter becomes zero under substitution, both sides again erase it. Otherwise both sides retain the constructor and the induction hypotheses identify its children. Normalization cannot turn any nonvariable constructor into literal zero. Formula binders require only the usual lifted substitution. Associativity follows by applying the displayed identity to successive raw substitutions. □

2.2 Formulas, clauses, assumptions, and contradiction

There is one numerical sort. Native formulas are

$$A ::= (s = t) \mid D_{x,y}B(x, y).$$

The distinct variables x, y are bound by D. Retained contexts are finite reusable lists of clauses

$$[\, A;\, y.B(y)\,], \qquad y \notin \mathrm{FV}(A).$$

Their ordinary readings are

$$\begin{aligned}(s = t)^\circ &= (s = t),\\ (D_{x,y}B)^\circ &= \exists x(B^\circ(x, 0) \to \forall y\, B^\circ(x, y)),\\ [\, A;\, y.B\,]^\circ &= A^\circ \to \forall y\, B^\circ(y).\end{aligned}$$

The equality on the right is actual equality. An ordinary assumption A abbreviates the constant clause $[\, 0 = 0;\, v.A\,]$, with v unused. The truth of $0 = 0$ is *derived*; it is not an initial axiom. The designated contradiction is $0 = S0$.

3 Exactly four inference families

Every supplied expression is canonical and well scoped. Bound names are freshened before substitution. Clauses are reusable; no linear or resource-sensitive reading is intended.

Calculation (C). For every counter, initializer, and body:

$$\frac{}{\Gamma \vdash R(Sn, a; u.h) = h[R(n, a; u.h)/u]}. \tag{C}$$

The predecessor iterate on the right is canonicalized; at $n \equiv 0$ it is Sa. This is one nullary computation schema.

Use (U).

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \qquad [\, A;\, y.B(y)\,] \in \Gamma. \tag{U}$$

The chosen clause and term t are part of the certificate.

Introduction (I).

$$\frac{\Gamma, B(t,\eta) \vdash B(t,Sz)}{\Gamma \vdash D_{x,y}B(x,y)}, \qquad \eta = \begin{cases} z, & B \text{ is literally an equation and } t \equiv 0, \\ 0, & \text{otherwise.} \end{cases} \tag{I}$$

Here z is fresh for Γ, t and all passive parameters of B. The single local assumption is the ordinary clause just defined. The literal test is part of the rule. In particular it is not a test of numerical equality to zero.

Substitutive witness discharge (O). Set $\Delta_b = \Gamma, [\, B(b,0);\, y.B(b,y)\,]$. Then

$$\frac{\Gamma \vdash D_{x,y}B \qquad \Delta_b \vdash s = t \qquad \Delta_b \vdash F(s)}{\Gamma \vdash F(t)}. \tag{O}$$

The parameter b is fresh for the outer context and the passive parameters of B. The *exported formula* $F(t)$ must have no free b. The template $F(v)$ may be any native formula, including a constant template; v is a distinguished template variable, not the witness b. The terms s, t and the template may mention b. Requiring all of them to be b-free would unnecessarily weaken the rule. All three premises are checked in their displayed scopes.

No unlisted primitives. There is no separate hypothesis rule, reflexivity axiom, equality transport rule, ordinary existential elimination, coverage axiom, separation axiom, logical connective, structural inference, or general numerical-substitution instruction. Weakening and cut are admissible proof transformations. The sole explicit computation schema is C; the equation/progress branch of I carries the induction strength.

4 The equality bootstrap is noncircular

Let

$$W_t := R(1, 0; u.t), \qquad J := (W_0 = 0), \qquad T := D_{x,y}J,$$

where u, x, y are absent from the indicated passive expressions. All these are finite canonical expressions. Rule C gives $W_t = t$, because the constant body ignores the predecessor iterate.

Lemma 4.1 (A closed paired theorem)**.** *The formula T has a proof using C and I only.*

Proof. Introduce $D_{x,y}J$ at witness 1. The matrix is independent of both bound variables, so the branch has goal J. Rule C proves J while ignoring the local assumption. In particular, this proof uses neither ordinary-assumption use nor reflexivity. □

Lemma 4.2 (Unrestricted formula transport)**.** *For every native template $F(v)$,*

$$\Gamma \vdash s = t, \quad \Gamma \vdash F(s) \quad \Longrightarrow \quad \Gamma \vdash F(t). \tag{Tr}$$

Proof. Use T as the first premise of O. Weaken the two supplied proofs into its opened clause context, without using the fresh witness. The exported formula is outer scoped. This argument does not use an equality law. □

Lemma 4.3 (Reflexivity and ordinary assumptions)**.** *For every term t, $\Gamma \vdash t = t$. Consequently ordinary-assumption use is admissible.*

Proof. Calculation gives $W_t = t$. Use it as both premises of Tr with template $F(v) = (v = t)$. Its source instance is $W_t = t$ and its target instance is $t = t$. To use an assumption A, first take this construction at $t = 0$; then U on $[\, 0 = 0;\ v.A\,]$ yields A. □

Lemma 4.4 (Ordinary equality and ordinary witness elimination)**.** *Symmetry, transitivity, term congruence, and the usual D-elimination rule are admissible.*

Proof. Given $s = t$, transport $s = s$ through $F(v) = (v = s)$ to obtain $t = s$. Given $s = t$ and $t = r$, transport the second equation along $t = s$ through $F(v) = (v = r)$ to obtain $s = r$. Given $s = t$, transport the reflexive equation $h(s) = h(s)$ through $F(v) = (h(s) = h(v))$ to get $h(s) = h(t)$.

For ordinary D-elimination, suppose its opened branch proves an outer-scoped formula A. In O choose the constant template $F(v) = A$ and use C's closed equation J as the local equality premise. The minor premise is precisely the supplied branch, and the exported formula is A. This recovery does not require a naming packet or any normalization of a selector. □

Structural proof operations. Weakening, exchange, contraction, and injective fresh renaming follow by induction on finite derivations. For cut of an ordinary assumption, replace a U node selecting its constant clause by the supplied proof; freshen local parameters first. These arguments preserve the literal witness test in I. We do not assume that arbitrary numerical substitution preserves that test: a free witness can become literal zero. The comparison below derives that admissibility indirectly.

5 Exact equivalence with the separate-rule calculus

Let $\mathcal{K}_{\mathrm{sep}}$ have the same terms, formulas, clauses, calculation, Use and Introduction, but replace O by two rules: full Tr as displayed above, and ordinary D-elimination. It has five families and no fixed reflexivity axiom.

Theorem 5.1 (Identity-language consequence equivalence)**.** *For every canonical clause context* Γ *and formula* A*,*

$$\Gamma \vdash_{\mathcal{K}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{K}_{\mathrm{sep}}} A.$$

The translations are effective on finite proofs and preserve the exact end sequent, including open numerical parameters.

Proof. Translate an O inference into Tr inside its opened witness scope, then ordinary D-elimination. Its side condition is exactly what the latter requires. Conversely, translate Tr using the closed theorem T and O, and ordinary D-elimination using constant F and local equality J. All other inferences translate directly. Induct on derivations, freshening bound eigenparameters when inserting the fixed auxiliary proofs. No search for a numerical witness or truth value is involved. □

5.1 Recovering the current canonical basis

For an unused numerical coordinate w, abbreviate

$$q_y B(y) := D_{w,y} B(y).$$

The following recoveries connect $\mathcal{K}$ to the current canonical Eq/Q/D basis of [2].

First, every case introduction is admissible, including an atomic matrix at witness zero. Let $c = W_0$. This is a nonzero *syntactic* term, and C proves $c = 0$. Given a case branch at witness zero,

start instead from $B(c,0)$, transport to $B(0,0)$, insert the supplied branch by ordinary-assumption cut, and transport its conclusion back to $B(c,Sz)$. I at c is a case introduction and has the desired conclusion $D_{x,y}B$. No induction is used in this repair.

Next, case introduction with an unused witness gives

$$\Gamma, B(0) \vdash B(Sz) \quad \Longrightarrow \quad \Gamma \vdash q_y B.$$

Ordinary D-elimination opens $q_y B$ as the clause $[\, B(0);\, y.B(y)\,]$. Together with a proof of $B(0)$, U gives $B(t)$ for any target term t. When B is literally an equation, I at the unused witness zero directly gives $q_y B$ from $\Gamma, B(z) \vdash B(Sz)$. Thus contextual equation induction and numerical cases for arbitrary native formulas are derived.

Conversely, a retained clause $[\, B(0);\, y.B(y)\,]$ proves the public formula $q_y B$: use case introduction at the unused witness 1; under its temporary assumption $B(0)$, use the retained clause at Sz. Ordinary-assumption use is already derived. Thus a source proof whose D-elimination branch retained a Q formula can be translated by first deriving its dummy-D representative from the opened clause and then cutting that proof into the translated branch. The two context representations are related by explicit derivations, not identified by convention.

Reflexivity, symmetry, and full transport recover the canonical basis's crossed equation transport, including its restricted sources. The crossed form of Q-elimination is recovered by symmetry at equation conclusions. Conversely, in that canonical basis a clause $[\, A;\, y.B\,]$ has the ordinary reading $A \to \forall y B$, represented by its established native macros. Its derived full equality transport and D-elimination validate O. Hence all clause sequents translate, and ordinary formula sequents have the same consequences in both directions.

6 The bi-interpretation with full first-order iPRA

6.1 What the inherited bridge actually supplies

The predecessor construction in [2] is made before assuming a full numerical selector, successor cancellation, arbitrary equality reflection, or induction on compound formulas. It supplies a fixed canonical term P with

$$P0 = 0, \qquad P(Sa) = a.$$

Its prelogic argument supports contextual finite cases, which are needed to avoid an illicit use of full logic during that construction. We inherit that work rather than asserting that a counter-aware recursor is already available.

The initialized scalar iterator is then a finite term:

$$\widehat{I}(n,a;u.h) = P\big(R(n,a;v.S(h[P(v)/u]))\big)\,. \tag{Init}$$

At zero, its computation is $P(Sa) = a$. At a successor, C exposes an outer successor which P removes. Thus

$$\widehat{I}(0,a;u.h) = a, \qquad \widehat{I}(Sn,a;u.h) = h[\widehat{I}(n,a;u.h)/u].$$

The scalar bridge then gives a faithful Boolean equality test, quantifier-free induction, pairing and projections, and a compiler for each fixed primitive recursive description. Those implications are explicitly proved in the included reference sources and are separately formalized for the scalar basis in the repository [4, 5].

6.2 The complete logical interface is represented

For clarity, the derived logic is not just an equational fragment. Put $C(v,a,b) = \widehat{I}(v,a;u.b)$. Define the gate at equations by

$$G_v(s=t) := (C(v,0,s) = C(v,0,t))$$

and extend it through D by $G_v(DB) = D(G_vB)$, freshening names. It has a provable zero instance and its successor instance is interderivable with the gated formula. For two equation matrices set

$$M_v(s=t, p=q) := (C(v,s,p) = C(v,t,q)).$$

Extend M through two D roots componentwise. When only one input has a D root, pad the other by $D_{x,y}G_y$ before descending. This recursion decreases the maximum binder depth. It yields $M_0(A,B) \dashv\vdash A$ and $M_{Sz}(A,B) \dashv\vdash B$.

Let $p(z) = \widehat{I}(z,0;u.C(u,1,0))$; its needed fixed values are $p(1) = 1$ and $p(2) = 0$. The ordinary connectives and quantifiers have the following native representatives, with fresh unused w:

$$\begin{aligned} A \Rightarrow_* B &:= D_{w,z}M_z(A,B), \\ A \wedge_* B &:= D_{w,z}G_zM_{p(z)}(A,B), \\ \exists_* x\, A(x) &:= D_{x,y}G_yA(x), \\ A \vee_* B &:= \exists_* x\, M_x(A,B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* D_{w,z}G_zA(z), \\ \bot_* &:= (0 = S0). \end{aligned}$$

Their usual intuitionistic introduction and elimination rules follow from cases, the gate endpoints, clause specialization, and ordinary D-elimination. Conjunction projections specialize at 1 and 2; disjunction uses the witness as a zero/successor selector. Coverage then follows from cases and reflexivity. From $0 = S0$, selector transport proves every equation, and external induction on formula syntax proves every native formula. No excluded middle for quantified formulas is introduced. The full details and both logical round trips are retained in the included inherited logical bridge [5].

Theorem 6.1 (Whole-domain bi-interpretation)**.** *The ordinary realization of $\mathcal{K}$ and full first-order* iPRA *are bi-interpretable. Both domain formulas are $x = x$, interpreted equality is actual equality, and both numerical composite isomorphisms have graph $x = y$. Both formula composites are provably equivalent to the original formulas. In particular the two theories are equiconsistent.*

Proof. Interpret each fixed R term in iPRA by primitive recursion with seed Sa and its displayed update. The zero-body canonicalization preserves its value. C is a defining equation; U is implication and universal elimination; I uses numerical coverage in case mode and equation induction under its assumed base in progress mode. O is equality substitution inside an existential-elimination scope.

In the reverse direction, use the predecessor, initialized iterator (Init), scalar compiler, and the logical macros above. The preceding section proves that their necessary rules are available in $\mathcal{K}$. This supplies all defining equations and quantifier-free induction of iPRA.

For numerical composites, work through each fixed finite primitive recursive description in dependency order. After its earlier subdescriptions have been identified, the description and its return translation have identical seeds and updates. Equation induction on their two trajectories proves their equality for every number in the whole domain. For an R body, pointwise body equality is lifted to iteration by the same trajectory argument; it is not assumed as a free intensional congruence rule. The interpretation of successor returns $I(0,Sa;u.0) = Sa$. Hence the numerical composite maps are the identity, not merely bijections of standard term values.

The established logical round trips identify the native D reading and its returned macro. Induction on formula construction then extends the numerical identities to both complete formula composites. The contradiction $0 = S0$ corresponds to falsum in each direction, giving equiconsistency by the effective proof translations. □

7 The complete checker and the simplicity comparison

7.1 Input and trust boundary

The selected module is `code/canonical_checker.py`. It accepts exactly a JSON object with `goal` and `proof`. The goal is a closed canonical Eq/D formula. Integer zero is numerical zero, negative integers are free eigenparameter levels, and positive integers are de Bruijn indices. An R node has length three, an equation has length two, and D has length one. Scope and grammar validation inspect every child before canonicalizing; malformed content in an erased zero-counter body is rejected.

The four instructions are `c`, `u`, `i`, and `e`. Clause use checks its antecedent proof. Discharge checks the D major, a local equality proof, a local template-instance proof, and the exact exported outer-scoped goal. Ordinary assumptions are represented by clauses with antecedent $0 = 0$. There is no preloaded reflexivity clause or imported theorem database.

All replacement terms are closed with respect to loose positive indices: their external parameters have negative levels. A replacement term is therefore inserted unchanged beneath binders; its own internal indices remain bound by its own R or D nodes. Template traversal still tracks binding depth and decrements indices when a binder is opened. This invariant is checked at the points where terms enter the kernel.

Proof and syntax recursion are structural. The only canonicalization erases a body at a literal-zero counter. A standard coding of finite trees therefore makes the certificate relation primitive recursive; no uniform numerical evaluator, normalization-totality axiom, or fuel field is required. Concrete Python recursion or memory limits can of course reject a sufficiently large valid finite certificate.

7.2 Measured results and fair comparisons

All measured sources are complete standalone modules, including JSON input, validation, binding operations, inference checks, and final-goal comparison. Actual module docstrings are removed; the AST is serialized by `ast.unparse` with a final LF. Token counts omit layout and comments. Gzip-9 with timestamp zero is informational.

Complete module	Bytes	Tokens	AST	Gzip
Matched separate five-family control	2318	726	809	889
Selected four-family calculus	2267	713	798	882

The reduction is 51 bytes (2.20%), 13 significant tokens (1.79%), and 11 AST nodes (1.36%). These are modest savings. The control and proposal share the same grammar, canonicalizer, binding representation, assumption representation, names, and I rule; only equality transport and witness elimination are separated or fused.

The incoming full-normalization Name checker is 2273 normalized bytes, 732 tokens, and 813 AST nodes as delivered. The numerical comparison 2267 versus 2273 changes several design decisions and is *not* an isolated measure of our mathematical change. The current compact 516-byte and

validated-public-goal 788-byte profiles are much smaller and use different certificate contracts. Our result is not a new corpus-wide minimum.

7.3 Executed checks

The canonical suite passed 14 positive and 18 adversarial cases, 14 exact-goal translations through the matched control and back, and 64 standalone checks across ordinary and optimized Python. The raw companion passed 14 positives, 16 rejections, two open sequents, 60 command-line checks, 14 control round trips, and five translations from the frozen raw CE5 checker. There were no unexpected outcomes.

The archive includes executable positive and adversarial certificates, an independently generated raw-scalar companion, and the matched canonical control. The final evidence file records the exact runs. The intended coverage is computation, reflexivity derived without an axiom, use of a retained induction hypothesis, compound-template transport, nested binding, ordinary witness elimination, and rejection of witness escape and ill-scoped zero-body contents. Finite tests validate these implementations; the general soundness, equivalence, and bi-interpretation claims rest on the proofs above and the stated inherited bridges.

8 A boundary result: unary congruence is insufficient

One tempting further simplification is to replace binary equality transport by the unary rule $s = t \Rightarrow h(s) = h(t)$. The following small counterinterpretation shows why that does not work for the initialized-scalar raw or zero-normal language, even when reflexivity, symmetry, computation, and equation induction are retained.

Proposition 8.1. *In that reduced Eq/Q/D calculus, transitivity of equality is not derivable. This remains true when $I(0, a; u.h)$ is definitionally a.*

Proof. Use $V = \{0, m, 2\}$, identified for distance purposes with $\{0, 1, 2\}$. Interpret zero as 0, successor as the constant 2, and

$$I(n, a; u.h) = \begin{cases} a, & n = 0, \\ m, & n \neq 0. \end{cases}$$

The body is ignored. Interpret the syntactic equation atom by the reflexive symmetric relation $R(s, t)$ meaning that their numerical distance is at most one. This is a truth assignment to judgments, not a structure using actual first-order equality. The contradiction $0 = S0$ is false.

Zero computation holds as identity. A successor iterator denotes m, which is related to every possible value of its right-hand side, so successor computation holds as a judgment. The numerical operation $F(n, a)$ above preserves R simultaneously in both arguments: when exactly one counter is zero the outputs are a and m; when both are zero use $R(a, a')$; otherwise both outputs are m. Thus every term template preserves R, validating unary congruence.

Each term function is either constantly m, or equals a payload $p \in \{0, 2, \text{one free variable}\}$ when all variables in a finite gate set are zero, and equals m otherwise. Structural induction proves this classification. Fixing all but one variable, the possible value triples are

$$(0, 0, 0), (m, m, m), (2, 2, 2), (0, m, 2), (0, m, m), (2, m, m).$$

Therefore an atom's one-variable truth set is empty, $\{0, m\}$, $\{m, 2\}$, or all of V.

Give Q and D their displayed Boolean readings. Q applied to an atom is true, false, or a finite disjunction of conditions that a free variable is nonzero. Indeed, if its newly bound variable is in a term's gate, the atom is true at both nonzero values; if it is only in a payload, the only obstruction to closure occurs when a finite set of other variables are all zero. Existential binding for D leaves such a disjunction alone or makes it true. A further Q or D is true. It follows that every native formula satisfies $B(0) \wedge B(2) \Rightarrow B(m)$ under fixed remaining parameters.

This property validates case introduction because every successor is 2. It also validates equation progress: from its base at 0 and its step at 0 obtain the value at 2, then the value at m. QE and DE are sound under their ordinary Boolean readings. Hypotheses and the other retained rules preserve truth as well.

Assign $x = 0, y = m, z = 2$. Both $x = y$ and $y = z$ are true, while $x = z$ is false. Thus the sequent expressing transitivity cannot be derived. □

Full successor-computation normalization is outside this obstruction: it would identify values which this interpretation only relates. Nor does the result exclude every different arithmetic presentation. It excludes this specific deletion in a precisely stated neighborhood of the current scalar core.

9 Further research questions

1. Can substitutive witness discharge be formalized as a small comparison module on top of the existing Lean clause development? The closed dummy-D proof makes its two translations especially direct.
2. Can its template be restricted to a term template, with compound transport recovered, while preserving a net saving in the complete checker? The permitted witness occurrences must be tracked carefully.
3. Can a guard independent of the supplied witness replace I's literal-zero test without undoing the source-size reduction?
4. Can the constancy-only presentation in the next appendix recover its Boolean equality split from a simpler arithmetic basis? The split must not be silently assumed during that bootstrap.
5. Can a common public grammar and malformed-input contract be fixed for the compact and strict interfaces? Only then would a lower bound or a global checker-size claim have an unambiguous target.

A A constant-recurrence rule in place of induction

This section gives a separate, complete arithmetic presentation. Its primitive inference rule has neither an induction hypothesis nor an arbitrary update function: it says that a term which is unchanged by successor has its initial value everywhere. The price is an explicit Boolean equality operation. That price is part of the presentation and must be counted in any simplicity comparison.

A.1 The complete arithmetic base

Use ordinary intuitionistic first-order logic with actual equality, on one numerical sort. Terms have the grammar

$$t ::= x \mid 0 \mid S(t) \mid \delta(t, t) \mid R(n, a; i, u.h).$$

The last constructor binds i, u in h only. Its body is finite earlier term syntax; it is not a variable ranging over numerical functions. All substitutions are capture avoiding. Put $1 = S0$ and $s \neq t = (s = t \to \bot)$. The nonlogical axioms are the universal closures of the following four displayed schemata:

$$R(0, a; i, u.h) = a, \tag{R0}$$

$$R(Sn, a; i, u.h) = h[n/i, R(n, a; i, u.h)/u], \tag{RS}$$

$$0 = 1 \longrightarrow \bot, \tag{Sep}$$

$$\big(s = t \wedge \delta(s, t) = 0\big) \ \vee \ \big(s \neq t \wedge \delta(s, t) = 1\big). \tag{Split}$$

The substitution in (RS) is simultaneous. No coverage axiom, successor-injectivity axiom, induction schema, or equality-reflection theorem is silently included.

The one additional inference schema is

$$\frac{\Gamma \vdash f(Sz) = f(z)}{\Gamma \vdash f(t) = f(0)} \quad \text{(Const)}, \tag{Const}$$

where $f(x)$ is a fixed numerical term template, z is fresh for Γ, t and the passive parameters of f, and the displayed premise contains no new numerical or equational assumption. The context Γ may contain arbitrary first-order formulas.

The underlying logic can be taken to be standard intuitionistic natural deduction: assumption; introduction and elimination for $\wedge, \vee, \to, \forall, \exists$; falsum elimination; equality reflexivity and substitution; and the usual eigenvariable conditions for $\forall$ introduction and $\exists$ elimination. Thus this presentation does not use a classical inference rule or a choice principle.

A.2 Elementary computations require no induction

Define

$$C(v, a, b) = R(v, a; i, u.b), \qquad a + b = R(b, a; i, u.Su), \qquad P(v) = R(v, 0; i, u.i).$$

Their displayed equations are direct instances of the computation axioms:

$$\begin{aligned} C(0, a, b) &= a, & C(Sv, a, b) &= b, \\ a + 0 &= a, & a + Sb &= S(a + b), \\ P0 &= 0, & P(Sv) &= v. \end{aligned}$$

In particular $a + 1 = Sa$ and successor is injective.

Every successor is different from zero. Indeed, if $Sv = 0$, applying $w \mapsto C(w, 0, 1)$ gives $1 = 0$, contradicting (Sep). This argument uses no case distinction on an arbitrary number.

The split axiom, together with $0 \neq 1$, gives

$$\delta(s, t) = 0 \vee \delta(s, t) = 1, \qquad \delta(s, t) = 0 \ \longleftrightarrow \ s = t. \tag{E}$$

For the reverse direction of the equivalence, eliminate the two disjuncts of (Split) under $s = t$; its second disjunct contradicts that assumption. For the forward direction, its second disjunct would give $0 = 1$. These are intuitionistic derivations.

A.3 Boolean failure terms for quantifier-free formulas

For every quantifier-free formula A, recursively define a term q_A, with value zero meaning that A holds:

$$\begin{aligned} q_{s=t} &= \delta(s,t), & q_{\bot} &= 1, \\ q_{A\wedge B} &= C(q_A, q_B, 1), & q_{A\vee B} &= C(q_A, 0, q_B), \\ q_{A\to B} &= C(q_A, q_B, 0). \end{aligned}$$

Negation is the abbreviation $A \to \bot$. Structural induction on the formula, using the two computations for C, proves

$$q_A = 0 \vee q_A = 1, \qquad q_A = 0 \iff A. \tag{B}$$

The induction here is a metatheoretic induction on a finite formula, not an arithmetic induction inside the proposed theory. In the implication case, split on q_A: when it is zero, A is available and $A \to B$ is equivalent to B; when it is one, $\neg A$ follows and the implication is true. Only the decidability already proved for the quantifier-free antecedent is used. No claim is made for formulas with quantifiers.

A.4 An accumulator turns progress into constancy

Suppose $A(n)$ is quantifier free and that the proposed theory has derivations

$$\Gamma \vdash A(0), \qquad \Gamma, A(z) \vdash A(Sz),$$

with the usual fresh-variable condition. Write $q(n) = q_{A(n)}$. Define the following finite terms:

$$\begin{aligned} e(n) &= C(q(n), 0, C(q(Sn), 1, 0)), \\ a(n) &= R(n, 0; i, u.u + e(i)), \\ F(n) &= a(n) + q(n). \end{aligned}$$

Here e and the addition operation are expanded into their finite term abbreviations before the outer R is formed. In particular, the definition of a does not presuppose a new function symbol or a new recursion principle.

The computations give

$$a(0) = 0, \qquad a(Sn) = a(n) + e(n).$$

The three possible pairs of Boolean values of $q(n), q(Sn)$ give the following equalities without an induction hypothesis:

$q(n)$	$q(Sn)$	$e(n)$	$F(Sn) = F(n)$
0	0	0	$a(n) = a(n)$
1	1	0	$Sa(n) = Sa(n)$
1	0	1	$Sa(n) = Sa(n)$

The remaining pair $(0, 1)$ is impossible: $q(n) = 0$ gives $A(n)$; the retained step derivation gives $A(Sn)$; and (B) then gives $q(Sn) = 0$, contradicting $0 \neq 1$. Thus, by finite disjunction elimination under Γ,

$$\Gamma \vdash F(Sz) = F(z).$$

Apply (Const) to obtain $F(t) = F(0)$. The proved base $A(0)$ gives $q(0) = 0$, so $F(0) = 0$. If $q(t) = 1$, then

$$0 = F(t) = a(t) + 1 = Sa(t),$$

contrary to successor separation. The Boolean disjunction for $q(t)$ therefore gives $q(t) = 0$, and hence $A(t)$.

Theorem A.1 (Recovery of contextual quantifier-free induction)**.** *The four axiom schemata* (R0)*,* (RS)*,* (Sep)*,* (Split) *and the rule* (Const) *derive*

$$\frac{\Gamma \vdash A(0) \qquad \Gamma, A(z) \vdash A(Sz)}{\Gamma \vdash A(t)}$$

for every quantifier-free A, under arbitrary retained first-order contexts. In particular they derive contextual induction for every literal equation.

No commutativity or associativity of addition was used in this proof. The only addition identities used are $a + 0 = a$ and $a + 1 = Sa$. This is why the arithmetic bootstrap does not conceal an induction argument in an algebraic rearrangement.

A.5 Coverage, primitive recursion, and both interpretations

Apply the recovered rule to the quantifier-free formula

$$A(n) \equiv n = 0 \vee n = S(Pn).$$

Its zero case is immediate. At a successor, $P(Sn) = n$ gives the second disjunct without an induction hypothesis. Thus numerical coverage is derived. Successor injectivity was already obtained from $P(Sn) = n$.

Every usual primitive recursive definition compiles directly into this counter-aware language: zero, successor, projections, and composition use the corresponding term operations, and

$$f(0, \vec{x}) = g(\vec{x}), \qquad f(Sn, \vec{x}) = h(n, f(n, \vec{x}), \vec{x})$$

is represented by

$$R\big(n, g(\vec{x}); i, u.h(i, u, \vec{x})\big),$$

after recursively compiling g, h. The computation equations are literal instances of (R0) and (RS). Together with successor separation, injectivity, and recovered quantifier-free induction, this gives an interpretation of full first-order iPRA.

Conversely, each fixed finite R term denotes a primitive recursive function in iPRA by ordinary primitive recursion on its counter. Its two defining equations hold by construction. Interpret δ by a Boolean equality characteristic. For an explicit construction, define

$$d(a, b) = R(b, a; i, u.Pu).$$

The subtraction equations and equation induction give

$$d(a, 0) = a, \quad d(a, Sb) = P(d(a, b)), \quad d(Sa, Sb) = d(a, b), \quad d(Sn, n) = 1.$$

For the third identity, induct on b at fixed a; the base is $P(Sa) = a$, and the step applies P to the preceding equality. For the fourth, induct on n using the third identity. Elementary equation inductions on the second argument of addition also give $0 + b = b$ and $Sa + b = S(a + b)$.

Quantifier-free induction on b, with a passive, proves

$$d(a, b) \neq 0 \;\longrightarrow\; a = b + d(a, b). \tag{Rec}$$

The base uses $0 + a = a$. For the step, put $r = d(a, b)$ and suppose $Pr = d(a, Sb) \neq 0$. Coverage on r gives $r = S(Pr)$: the zero case contradicts $Pr \neq 0$, and the successor case uses the equation for P. In particular $r \neq 0$, so the preceding induction hypothesis gives

$$a = b + r = b + S(Pr) = Sb + Pr.$$

This establishes the new instance of (Rec).

Now define

$$Z(a) = C(a, 1, 0), \qquad U(a) = C(a, 0, Z(Pa)), \qquad E(a, b) = C\big(b, Z(a), U(d(a, Pb))\big).$$

Coverage and successor separation give $Z(a) \in \{0, 1\}$ and $Z(a) = 1 \leftrightarrow a = 0$. They similarly give $U(a) \in \{0, 1\}$ and $U(a) = 1 \leftrightarrow a = 1$. If $b = 0$, $E(a, b) = 1$ is equivalent to $a = 0$. If $b = Sn$, $E(a, b) = 1$ is equivalent to $d(a, n) = 1$. The latter condition implies $a = n + 1 = Sn = b$ by (Rec); conversely $a = Sn$ gives $d(Sn, n) = 1$. Thus $E(a, b)$ is Boolean and equals one exactly when $a = b$. The fixed primitive recursive term

$$\delta_*(a, b) = Z(E(a, b))$$

is therefore Boolean and equals zero exactly when $a = b$. This derivation uses only quantifier-free inductions after they have already been recovered; it is not part of the induction-free bootstrap.

Consequently the chosen equality characteristic satisfies (Split). The rule (Const) is iPRA-admissible: under its recurrence premise, equation induction on $f(n) = f(0)$ gives the conclusion. All domains remain the whole numerical domain and equality is interpreted as actual equality.

For completeness, the interpretation composites require more than agreement in the standard natural numbers. In the proposed theory, recovered equation induction proves uniqueness of every fixed primitive recursive defining recursion, under arbitrary parameters. By structural induction on term syntax, it therefore proves that the round trip of every R term has the same value as the original. The same argument in iPRA treats the round trip of each standard primitive recursive function symbol. For the additional primitive δ, both δ and the translated δ_* satisfy (Split) in the proposed theory. Case analysis on the first split shows their equality: if $a = b$, both are zero; if $a \neq b$, both are one. Hence

$$\delta(a, b) = \delta_*(a, b)$$

is provable there. Equality congruence and structural induction on formulas now give both formula round trips. The numerical isomorphisms have graph $x = y$.

Theorem A.2 (Exact scope of the result)**.** *The presentation in this section is whole-domain bi-interpretable with full first-order iPRA, with both composite numerical interpretations provably the identity. It is therefore equiconsistent with iPRA. The universal statements are written mathematical proofs, not a new proof-assistant formalization.*

A.6 What this does and does not simplify

The primitive arithmetic inference now concerns a single numerical term and an unconditional identity-update recurrence. It has no discharged induction hypothesis, no Boolean accumulator in its rule text, and no arbitrary update template. The accumulator is an untrusted derived proof construction.

The separate equality split is essential to the proof given here. Replacing it by an assertion that an internally defined equality test is faithful, without proving that assertion in the induction-free base, would be circular. Likewise, for a tail-recursive iterator, $d(a, Sb) = d(Pa, b)$ does not by itself imply $d(a, Sb) = P(d(a, b))$: the missing commutation is a fusion theorem.

Compared with the repository's scalar basis, this presentation adds a binary term constructor, a stronger primitive equality split in place of primitive coverage, and a second local binder for the recursor. It removes arbitrary induction from the primitive inference interface. No claim is made that those changes reduce the complete checker-size metric, surpass the existing compact profiles, or establish an absolute minimum. The unanswered next question is whether the equality split can be derived from a sufficiently small recursion base and (Const) alone.

B Reproduction and artifact map

Run `python3 reproduce.py` in the extracted directory. It replays the included validation scripts and regenerates complete-module metrics. Run `python3 build_pdf.py` to rebuild this PDF with pdfLaTeX; no shell escape is needed. The exact selected checker is included below and as an executable file. Supporting source snapshots retain the repository's MIT No Attribution license.

`report.tex`, `report.pdf`
: The new construction, proofs, explicit limits, and the constancy appendix.

`code/`
: Complete selected and matched-control checkers.

`validation/`, `examples/`, `evidence/`
: Untrusted certificate producers, executable checks, measured sizes, and results from the delivered run.

`reference/`
: Pinned current README, canonical-recursors proof source, logical and ordinary bridges, source metadata, and license. These are attributed prerequisites; their historical novelty claims are not claims of this report.

`CLAIMS.json`, `FORMAT.txt`, `README.txt`
: Machine-readable scope, the exact certificate contract, and entry points.

C Complete selected acceptance module

```
"""Canonical successor-seeded iPRA with substitutive witness discharge.

Terms: integer 0; negative free levels; positive bound indices; [n,a,h].
R(0,a;u.h) is canonically [0,a,0], and S(a) abbreviates [0,a,0].
Formulas: [s,t] or [B], with D binding x at index 2 and y at index 1.
Four instructions: c (successor calculation), u (clause use),
i (paired introduction), e (substitutive witness discharge).
No reflexivity axiom and no execution of iterator bodies by normalization.
Input: exactly {'goal': closed_formula, 'proof': finite_proof_tree}.
"""
import json
import sys


def need(condition):
    if not condition:
        raise ValueError('invalid certificate')


def walk(a, f, d=0, form=None):
    if type(a) is int:
        need(not form)
        return f(a, d)
    need(type(a) is list)
    size = len(a)
    need(form is None or size in ((1, 2) if form else (3,)))
    a = [walk(b, f, d + (2 if size == 1 else int(size == 3 and i == 2)), size == 1)
         for i, b in enumerate(a)]
    if size == 3 and a[0] == 0:
        a[2] = 0
    return a


def syntax(a, n, form=False, depth=0):
    def variable(i, d):
        need(-n <= i <= d)
        return i
    need(walk(a, variable, depth, form) == a)


def sub(a, t, depth=0):
    return walk(a, lambda i, d: t if i == d + 1 else i - (i > d + 1), depth)


H = [0, 0]


def prove(a, p, g, n):
    syntax(a, n, True)
    match p:
        case ['c']:
            need(len(a) == 2)
            counter, seed, body = a[0]
            need(type(counter) is list and counter[0] == 0)
            prev = [counter[1], seed, body if counter[1] else 0]
            need(a[1] == sub(body, prev))
        case ['u', j, t, premise]:
            need(type(j) is int and 0 <= j < len(g))
            syntax(t, n)
            base, body = g[j]
            need(a == sub(body, t))
            prove(base, premise, g, n)
        case ['i', t, branch]:
            need(len(a) == 1)
            syntax(t, n)
```

```
            body = sub(a[0], t, 1)
            hyp = sub(body, -n - 1 if len(body) == 2 and t == 0 else 0)
            prove(sub(body, [0, -n - 1, 0]), branch, g + [(H, hyp)], n + 1)
        case ['e', body, fml, s, t, major, eq, branch]:
            prove([body], major, g, n)
            body = sub(body, -n - 1, 1)
            local = g + [(sub(body, 0), body)]
            syntax(fml, n + 1, True, 1)
            prove([s, t], eq, local, n + 1)
            prove(sub(fml, s), branch, local, n + 1)
            need(a == sub(fml, t))
        case _:
            raise ValueError('invalid proof')


def check(data):
    need(type(data) is dict and set(data) == {'goal', 'proof'})
    prove(data['goal'], data['proof'], [], 0)


if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except Exception:
        print('REJECT')
        sys.exit(1)
    print('ACCEPT')
```

References

[1] Vladimir Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*. Pinned commit `17898ea7e6a`, current README and incoming archive inventory. https://github.com/VladimirReshetnikov/ProveIt/tree/17898ea7e6a1300dfdcad71040d5daeed67916d4/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] ProveIt research package, *Canonical recursors: iPRA with one explicit computation schema*, `Research/canonical_recursors/`. The complete pinned report source is included under `reference/`.

[3] ProveIt incoming package, *iPRA with equality at witness discharge*, `docs/incoming/ipra_witness_discharge_2026-11-11.zip`. Three families, fixed reflexivity clause, and full normalization.

[4] ProveIt, `Lean/ConciseIPRA/NativeBiInterpretation.lean`, `NativeClausal.lean`, and `NativeUniformClauses.lean`, at the pinned commit. Deep embeddings and the existing scalar/native comparison.

[5] ProveIt, `Research/certified_zero/reference/`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`. Attributed source files are included.